

Joins: combining tables

How do I answer a question that needs two tables?

BY THE END OF TODAY, YOU CAN

Class objectives

-
- 01 Say what one row is; predict row counts
 - 02 Explain why tables are split
 - 03 Run the Five Checks before a join
 - 04 Write INNER and LEFT JOIN; say which rows each keeps
 - 05 Audit a join and fix an inflated total
-

Warm-up

Five questions on D04, ungraded. The fifth looks ahead to joins.

Suggested time • 10 minutes

Review: D04 before joins

Every join changes what one row is. First, D04 in one query.

D04 in one query

Say what one row is, predict the count, then run.

Suggested time • 20 minutes

[labs/D06/01-Evr-D04-Review/README.md](#)

One row of orders is one ____?

Predict both numbers first. Equal counts mean order_id is unique.

```
SELECT COUNT(*) AS order_rows,  
       COUNT(DISTINCT order_id) AS different_orders  
FROM nakheel.orders;
```

→ 3,000 · 3,000: one row, one order

Revenue by channel: what will this print?

Read the error aloud: it says exactly what is missing.

```
SELECT channel, SUM(order_total) AS revenue  
FROM nakheel.orders;
```

SELECT list expression references column channel which is neither grouped nor aggregated

Every SELECT column is grouped or inside a function.

One row is one channel

WHERE keeps the completed rows first, then **GROUP BY** makes one row per channel.

```
SELECT channel,  
       COUNT(*) AS completed_orders,  
       ROUND(SUM(order_total), 2) AS revenue  
FROM nakheel.orders  
WHERE status = 'completed'  
GROUP BY channel  
ORDER BY channel;
```

→ **online 1,008 · 110,685.50 · store 1,763 · 205,338.00**

Add the size band as a CASE

One row is one channel and band: $2 \times 3 = 6$ rows.

```
SELECT channel,  
       CASE  
         WHEN order_total >= 200 THEN 'large'  
         WHEN order_total >= 50 THEN 'medium'  
         ELSE 'small'  
       END AS size_band,  
       COUNT(*) AS completed_orders,  
       ROUND(SUM(order_total), 2) AS revenue  
FROM nakheel.orders  
WHERE status = 'completed'  
GROUP BY channel, size_band  
ORDER BY channel, revenue DESC;
```

→ 6 rows

online large
107 · 52,844.50

store large
210 · 102,848.00

Only the groups worth more than 50,000

WHERE or **HAVING**? The test is on a group's total, which exists only after GROUP BY.

```
-- same SELECT, FROM, WHERE and GROUP BY as query 4
HAVING SUM(order_total) > 50000
ORDER BY revenue DESC;
```

channel	size_band	revenue
store	large	102,848.00
store	medium	82,422.50
online	large	52,844.50

→ 3 rows

WHERE keeps rows

WHERE tests one row at a time. Order 1002 is cancelled, so it is gone before any grouping.

```
FROM D03.orders → WHERE status = 'completed' → GROUP BY customer_id → HAVING COUNT(*) > 1
```

D03.orders · 5 rows

order_id	customer_id	status
1001	C-204	completed
1002	C-118	cancelled
1003	C-204	completed
1004	C-401	completed
1005	C-118	completed



after WHERE · 4 rows

order_id	customer_id	status
1001	C-204	completed
1003	C-204	completed
1004	C-401	completed
1005	C-118	completed

GROUP BY folds each group into one row

Every C-204 row goes into one C-204 row, wherever it sat. COUNT(*) counts them.

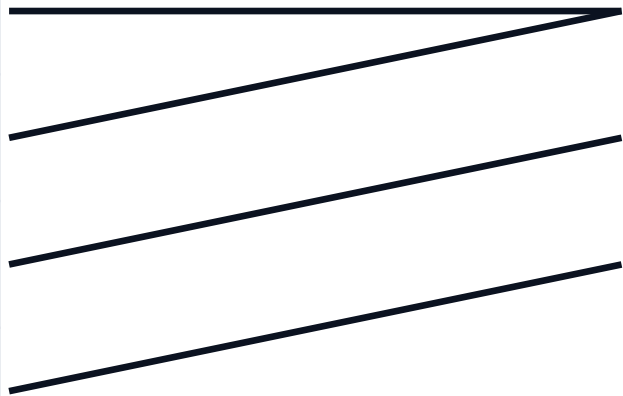
FROM D03.orders → WHERE status = 'completed' → **GROUP BY customer_id** → HAVING COUNT(*) > 1

after WHERE · 4 rows

order_id	customer_id	status
1001	C-204	completed
1003	C-204	completed
1004	C-401	completed
1005	C-118	completed

after GROUP BY · 3 rows

customer_id	order_count
C-204	2
C-401	1
C-118	1



HAVING tests each group

HAVING runs after GROUP BY, so it can test COUNT(*). One customer is left.

FROM D03.orders → WHERE status = 'completed' → GROUP BY customer_id → **HAVING COUNT(*) > 1**

after GROUP BY · 3 rows

customer_id	order_count
C-204	2
C-401	1
C-118	1



after HAVING · 1 row

customer_id	order_count
C-204	2

HAVING COUNT(*) > 1 keeps a group. **WHERE** keeps a row.

PART 02

Why the data is split, and how it joins back

D04's open question needs two tables at once.

Why the data is split

One customer, many orders, and why the name is stored only once.

Suggested time • 10 minutes

[labs/D06/02-Ins-Why-Tables-Split/README.md](#)

WHAT IF

If every order carried the customer's city

Lina moves from Amman to Aqaba. How many rows do you change?

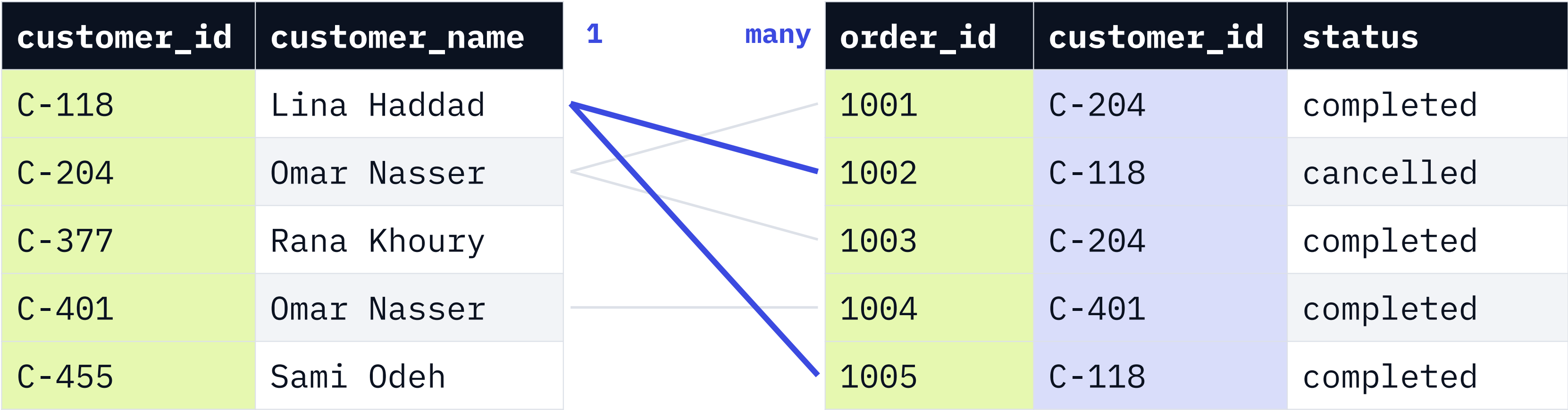
order_id	status	customer_name	city
1001	completed	Omar Nasser	Irbid
1002	cancelled	Lina Haddad	Amman
1003	completed	Omar Nasser	Irbid
1004	completed	Omar Nasser	Irbid
1005	completed	Lina Haddad	Amman

One customer, many orders

C-118 has two orders. Each order has exactly one customer.

Primary key

Foreign key



DEFINITION

A join puts the columns of two tables side by side for one query, matching rows on a key; neither table changes.

The Five Checks

Run them out loud before every join this week, and in Assignment 1.

Suggested time • 8 minutes

[labs/D06/02-Ins-Why-Tables-Split/README.md](https://github.com/DataCamp/labs/blob/main/D06/02-Ins-Why-Tables-Split/README.md)

The Five Checks before a join

-
- 01 What is one row in each table?
 - 02 Which key joins them?
 - 03 Is the key unique on the “one” side?
 - 04 How many rows should come out?
 - 05 What happens to rows with no match?
-

Is the key unique on the one side?

Equal counts mean every customer_id appears once in customers.

```
SELECT COUNT(*) AS customer_rows,  
       COUNT(DISTINCT customer_id) AS different_ids  
FROM D03.customers;
```

→ 5 · 5: unique

INNER JOIN and LEFT JOIN on D03

Predict 5 rows first. Checkpoint: C-377 and C-455 appear only in the LEFT JOIN.

Suggested time • 22 minutes

[labs/D06/03-Evr-Inner-and-Left/README.md](#)

AS o gives the table a short name

Both tables have customer_id, so **o.** and **c.** say which one.

```
SELECT o.order_id, o.status, c.customer_name, c.city
FROM D03.orders AS o
INNER JOIN D03.customers AS c
    ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

INNER JOIN keeps the rows that match

ON names the key on each side. Every order finds one customer.

```
SELECT o.order_id, o.status, c.customer_name, c.city
FROM D03.orders AS o
INNER JOIN D03.customers AS c
ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

→ 5 rows

customers LEFT JOIN orders

LEFT JOIN keeps every customer

7 rows. C-377 and C-455 have no orders, so their order columns are NULL.

customer_id	customer_name	order_id	status
C-118	Lina Haddad	1002	cancelled
C-118	Lina Haddad	1005	completed
C-204	Omar Nasser	1001	completed
C-204	Omar Nasser	1003	completed
C-377	Rana Khoury	NULL	NULL
C-401	Omar Nasser	1004	completed
C-455	Sami Odeh	NULL	NULL

customers INNER JOIN orders

INNER JOIN keeps only the matches

5 rows. C-377 and C-455 have nothing to match, so INNER JOIN drops them.

customer_id	customer_name	order_id	status
C-118	Lina Haddad	1002	cancelled
C-118	Lina Haddad	1005	completed
C-204	Omar Nasser	1001	completed
C-204	Omar Nasser	1003	completed
C-377	Rana Khouiry	NULL	NULL
C-401	Omar Nasser	1004	completed
C-455	Sami Odeh	NULL	NULL

← dropped: no order

← dropped: no order

HOW MANY ROWS?

Every match is one line

Each order points to one customer. Draw a line for each match, then count the lines.

customers

C-118	Lina Haddad
C-204	Omar Nasser
C-377	Rana Khoury
C-401	Omar Nasser
C-455	Sami Odeh

orders

1001	C-204
1002	C-118
1003	C-204
1004	C-401
1005	C-118

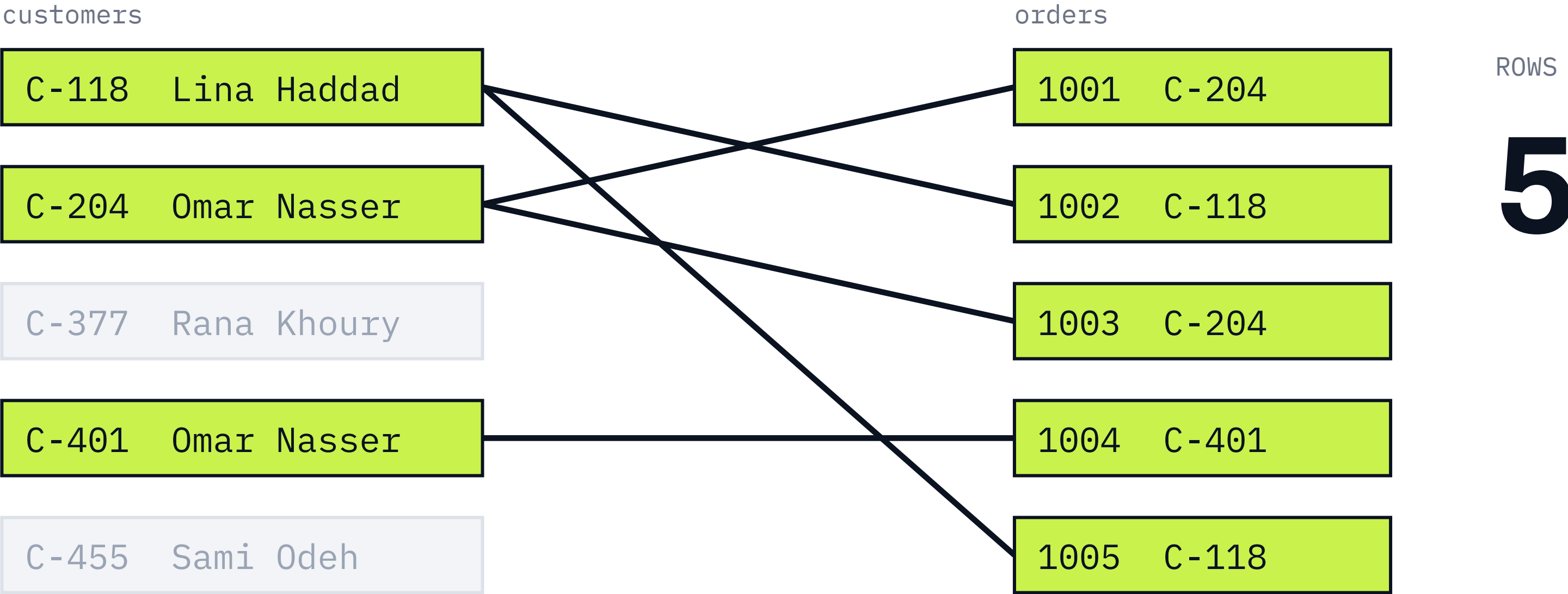
ROWS



HOW MANY ROWS?

INNER JOIN returns one row per line

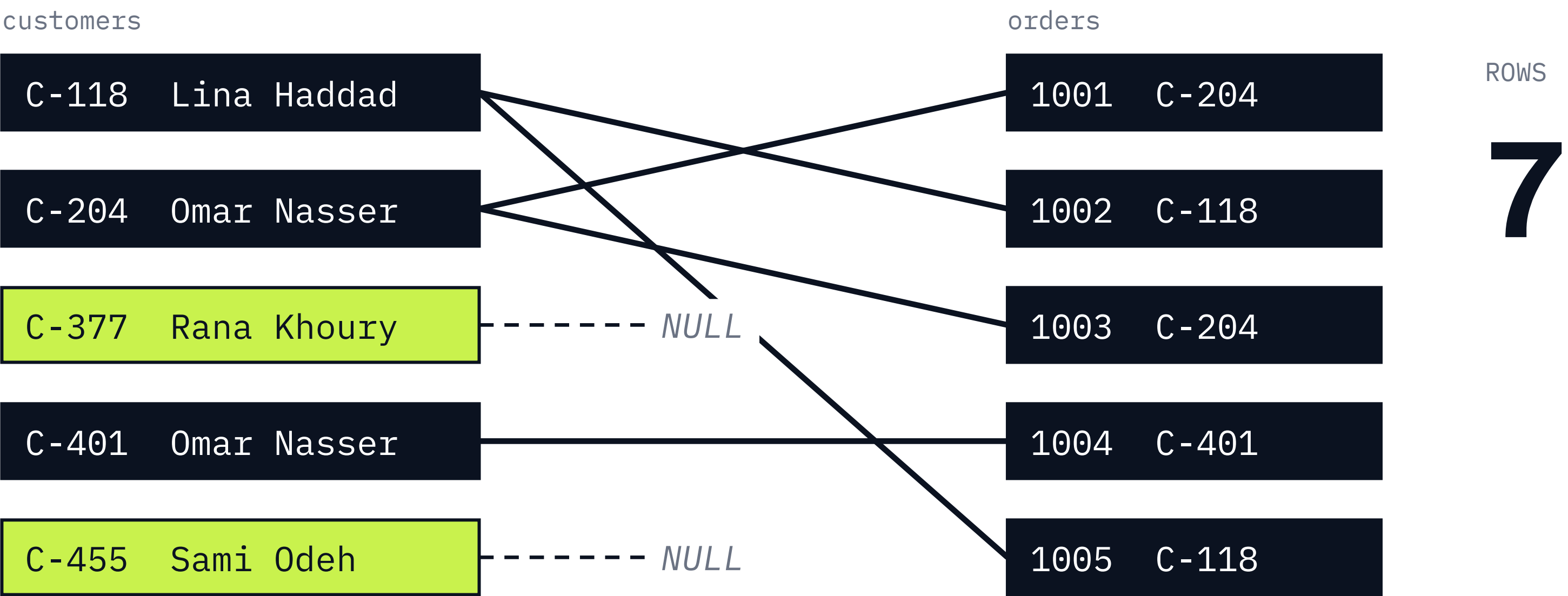
5 lines, 5 rows. C-377 and C-455 have no line, so INNER JOIN leaves them out.



HOW MANY ROWS?

LEFT JOIN adds the customers with no line

Each left row with no line comes back once, with NULL for the order: $5 + 2 = 7$.

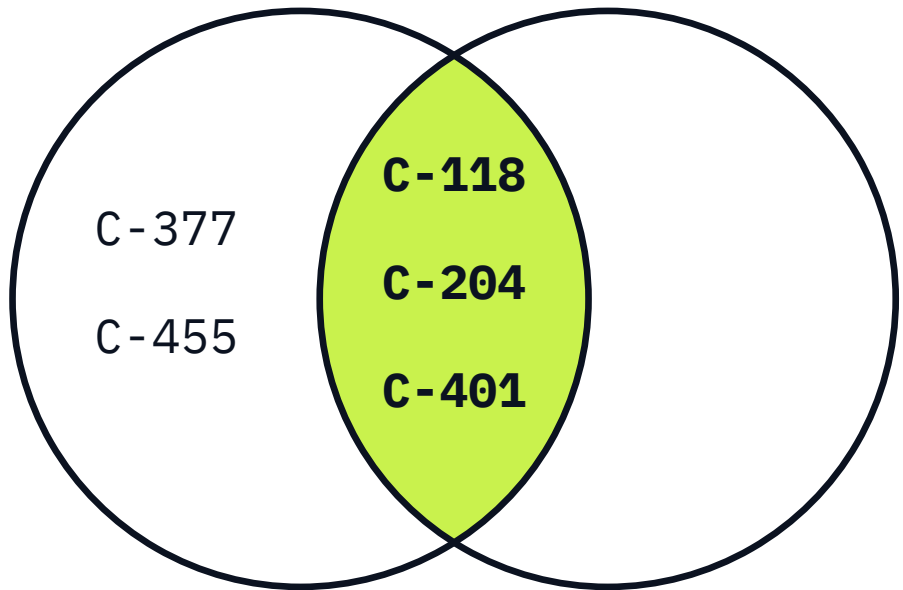


INNER JOIN: only keys found in both tables

D03.customers

customer_id	customer_name
C-118	Lina Haddad
C-204	Omar Nasser
C-377	Rana Khoury
C-401	Omar Nasser
C-455	Sami Odeh

customers



orders

D03.orders

order_id	customer_id
1001	C-204
1002	C-118
1003	C-204
1004	C-401
1005	C-118

```
SELECT c.customer_id, c.customer_name,
       o.order_id
FROM D03.customers AS c
INNER JOIN D03.orders AS o
ON c.customer_id = o.customer_id;
```

result · 5 rows

customer_id	customer_name	order_id
C-118	Lina Haddad	1002
C-118	Lina Haddad	1005
C-204	Omar Nasser	1001
C-204	Omar Nasser	1003
C-401	Omar Nasser	1004

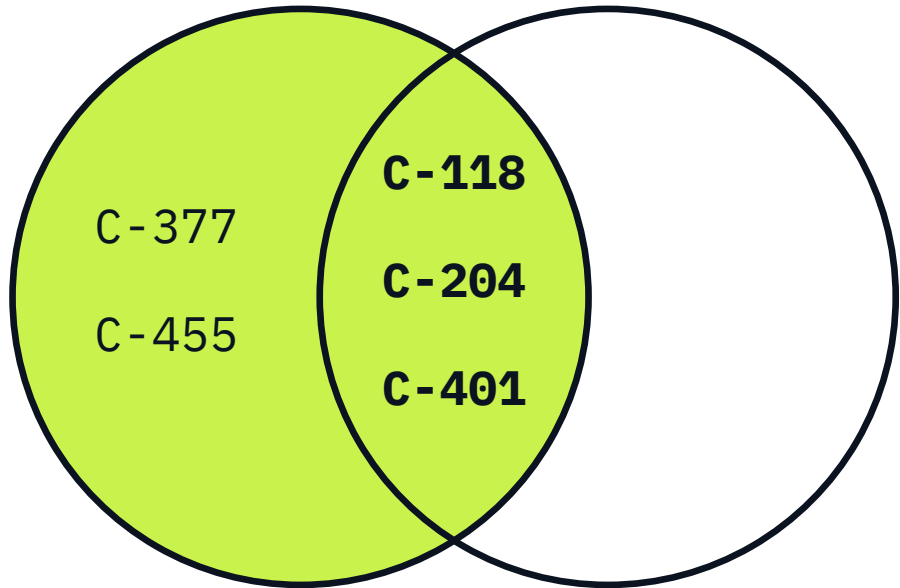
Rows out = the matches: 5.

LEFT JOIN: every customer, matched or not

D03.customers

customer_id	customer_name
C-118	Lina Haddad
C-204	Omar Nasser
C-377	Rana Khoury
C-401	Omar Nasser
C-455	Sami Odeh

customers



orders

D03.orders

order_id	customer_id
1001	C-204
1002	C-118
1003	C-204
1004	C-401
1005	C-118

```
SELECT c.customer_id, c.customer_name,
       o.order_id
FROM D03.customers AS c
LEFT JOIN D03.orders AS o
ON c.customer_id = o.customer_id;
```

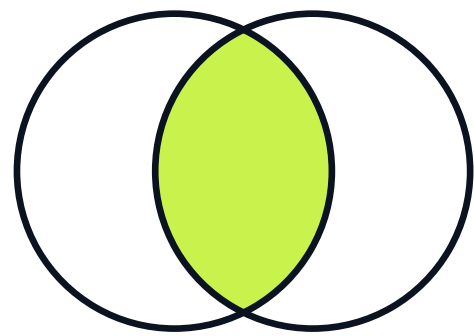
Rows out = matches + customers with no order: **5 + 2 = 7.**

result · 7 rows

customer_id	customer_name	order_id
C-118	Lina Haddad	1002
C-118	Lina Haddad	1005
C-204	Omar Nasser	1001
C-204	Omar Nasser	1003
C-377	Rana Khoury	NULL
C-401	Omar Nasser	1004
C-455	Sami Odeh	NULL

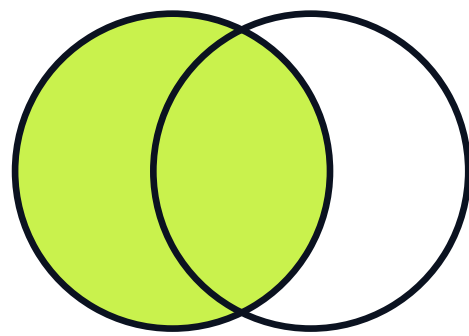
Two joins today, the rest on D07

INNER and LEFT JOIN are the two you will write most. RIGHT, FULL, UNION and CROSS JOIN come on D07.



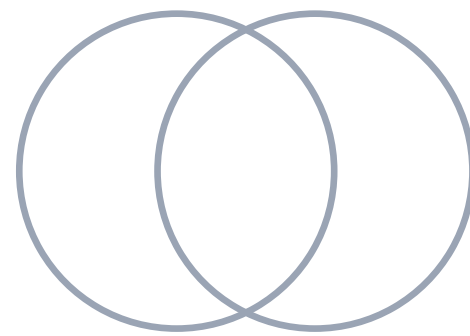
INNER JOIN

now



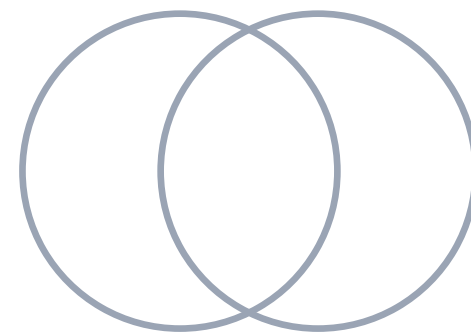
LEFT JOIN

now



RIGHT JOIN

D07



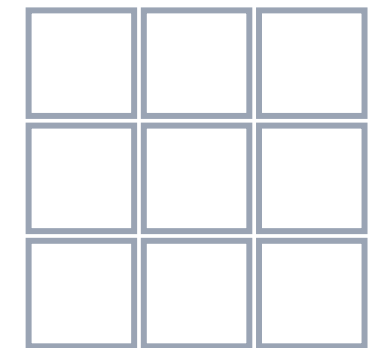
FULL OUTER

D07



UNION

D07



CROSS JOIN

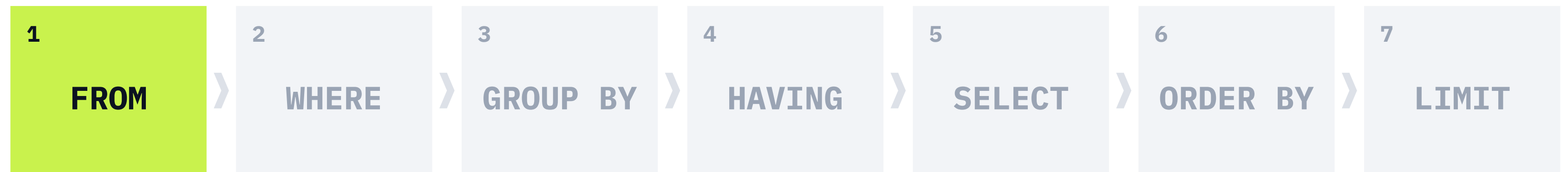
D07

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT

BIGQUERY RUNS



JOIN is part of **FROM**: the joined rows exist before WHERE runs.

Audit every join

Rows, distinct keys and the total, before and after.

How many completed orders?

The same question, before and after a join.

Suggested time • 10 minutes

[labs/D06/04-Evr-Count-Completed-Orders/README.md](#)

How many completed orders?

On orders

4

One row is one order

After joining order_items

10

One row is one product on one order

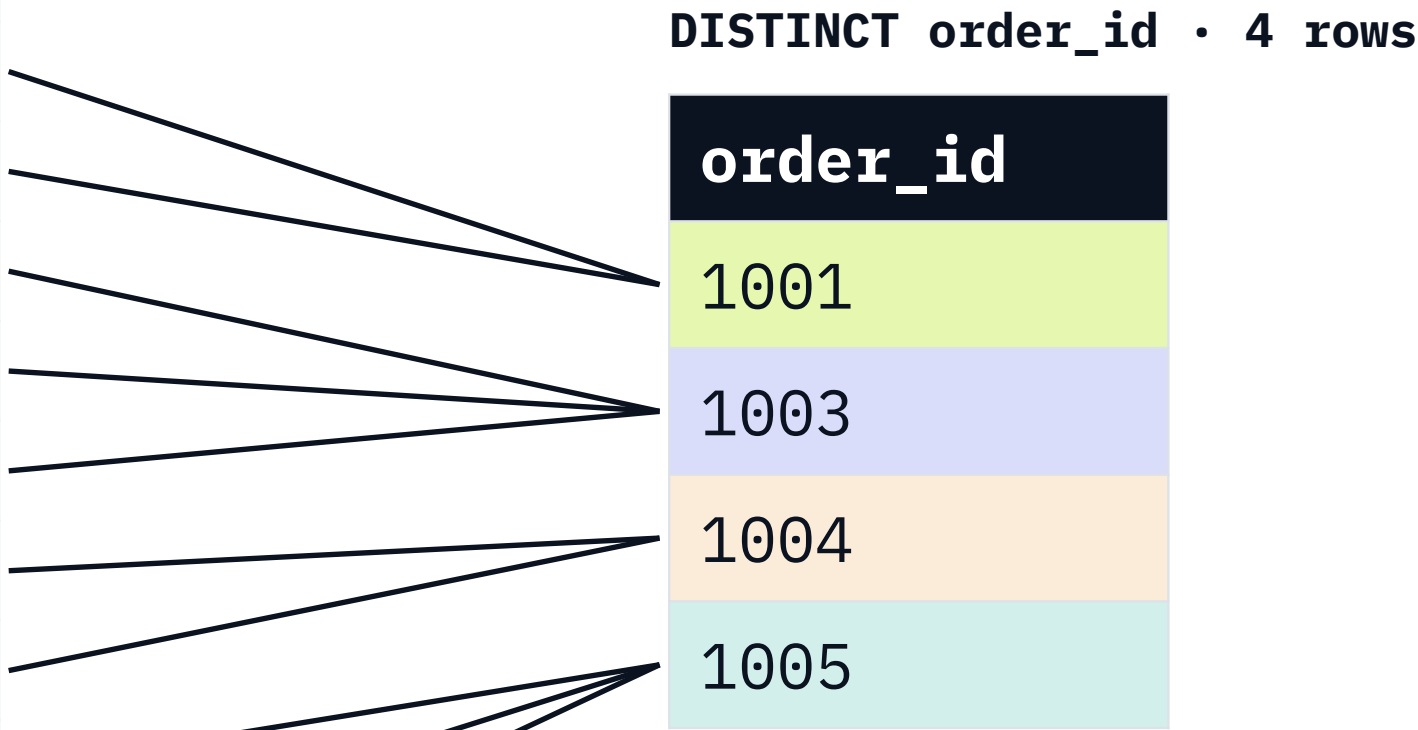
COUNT(DISTINCT o.order_id) after the join: 4 again.

Ten rows, four different orders

COUNT(DISTINCT o.order_id) after orders JOIN order_items: each order is counted once, however many lines it has.

completed orders JOIN order_items · 10 rows

order_id	item_id
1001	9001
1001	9002
1003	9005
1003	9006
1003	9007
1004	9008
1004	9009
1005	9010
1005	9011
1005	9012



COUNT(DISTINCT o.order_id)

4

COUNT(*) gives 10

Any item priced 20 or more?

Rows after the join

3

Order 1004 has two, so it counts twice

Different orders

2

Orders 1003 and 1004

COUNT(DISTINCT o.order_id) answers it: 2.

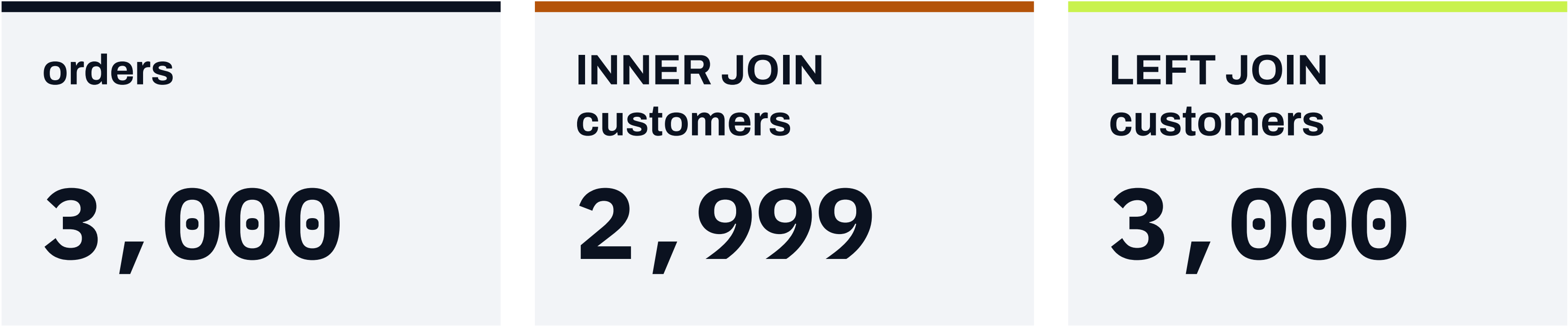
Join Nakheel orders to customers

3,000 orders. Count them again after INNER JOIN and after LEFT JOIN.

Suggested time • 10 minutes

[labs/D06/05-Evr-Orders-to-Customers/README.md](https://github.com/DataCamp/labs/blob/main/D06/05-Evr-Orders-to-Customers/README.md)

INNER JOIN drops one order



 Primary key  Foreign key

order_id	customer_id	order_date	status	order_total
101664	C-0999	2025-11-14	completed	938.00

C-0999 is not in customers.

STUDENT DO

Activity: Audit the join

In this activity, you will join Nakheel orders to their order lines and find which number the join changed.

Suggested time

15 minutes

[labs/D06/06-Stu-Join-Audit/README.md](#)

Activity: Audit the join

INSTRUCTIONS

1. On orders: rows, different orders, SUM(order_total).
 2. The same three after orders INNER JOIN order_items.
 3. After the join, sum qty * price instead.
- First, the Five Checks: which is the “one” side?

HINT

order_total is stored once per order. How often does it appear after the join?

BONUS

Which order had its total copied onto the most rows?

Time's up. Let's review.

Audit the join

Solved: `labs/D06/06-Stu-Join-Audit/Solved/join_audit.sql`

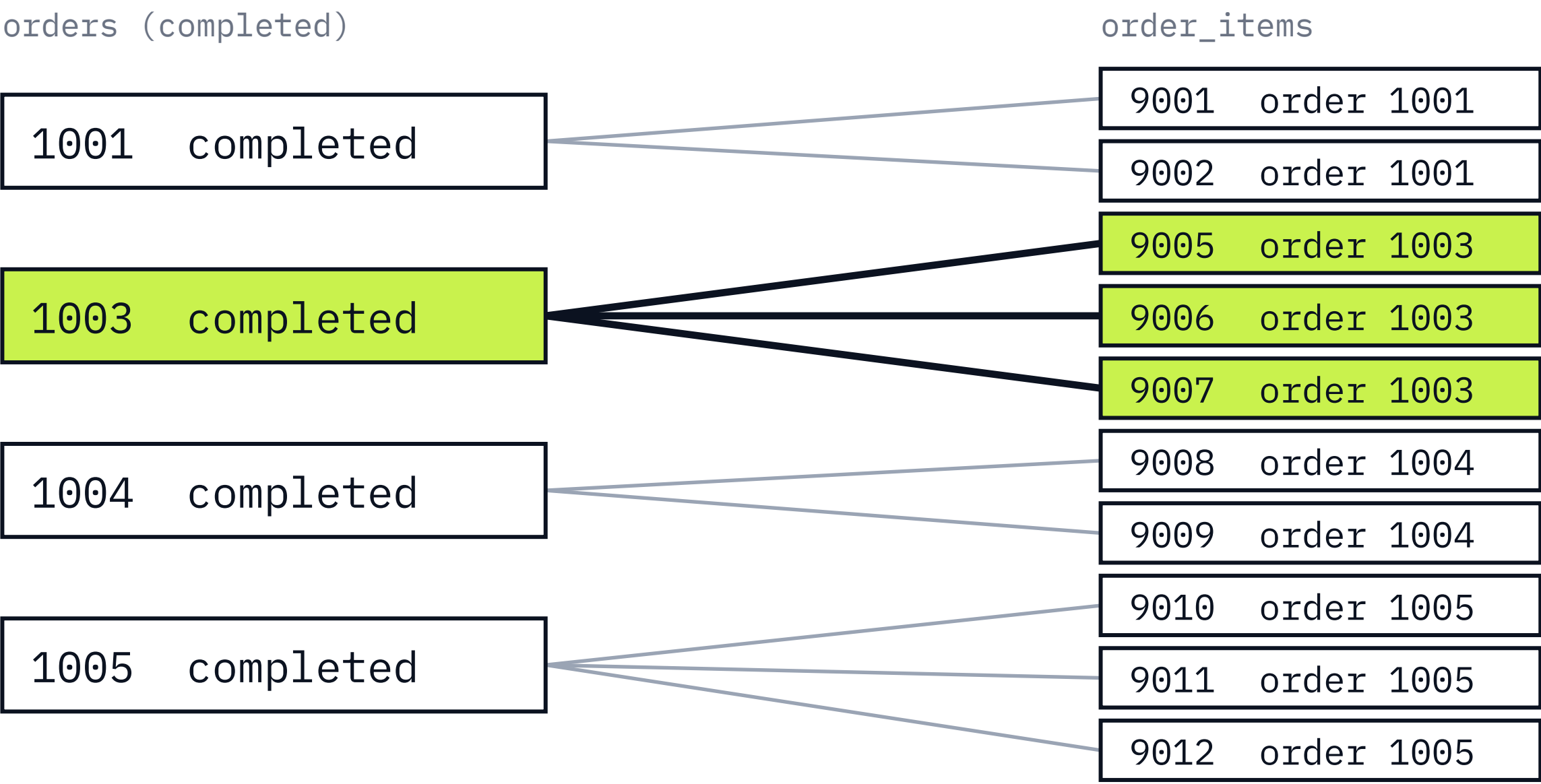
Which number did the join change?

SUM(i.qty * i.price) after the join: 340,370.00. The money is all there, summed at the wrong grain.

	Rows	Different orders	SUM(order_total)
orders	3,000	3,000	340,370.00
orders JOIN order_items	7,516	3,000	1,067,249.00

Order 1003 has three lines

4 completed orders, 10 lines. Order 1003 and its status come back three times.



ROWS

4 → 10

orders → joined rows
1003 appears 3 times

DEFINITION

Fan-out is what happens when a join to the “many” side copies each “one”-side row, and every amount stored on it.

Fix it, then answer D01's question

Sum each amount where it lives, then three tables for D01's question.

Sum each amount where it lives

Two fixes for the inflated total, and one rule.

Suggested time • 15 minutes

[1abs/D06/07-Ins-Sum-Where-It-Lives/README.md](https://github.com/101labs/D06/07-Ins-Sum-Where-It-Lives/README.md)

INFLATED

order_total, summed after the join

order_total lives on orders. The join copied it onto every line.

```
SELECT o.channel,  
       ROUND(SUM(o.order_total), 2) AS revenue_inflated  
FROM nakheel.orders AS o  
INNER JOIN nakheel.order_items AS i  
  ON o.order_id = i.order_id  
WHERE o.status = 'completed'  
GROUP BY o.channel  
ORDER BY o.channel;
```

→ **online 351,733.50 · store 637,238.00**

FIX 1

Sum the amount stored on each line

qty * price lives on order_items, the grain of the joined rows.

```
SELECT o.channel,  
       ROUND(SUM(i.qty * i.price), 2) AS revenue  
FROM nakheel.orders AS o  
INNER JOIN nakheel.order_items AS i  
  ON o.order_id = i.order_id  
WHERE o.status = 'completed'  
GROUP BY o.channel  
ORDER BY o.channel;
```

→ **online 110,685.50 · store 205,338.00**

Sum an amount at the table where it lives, or
prove the join did not change that table's grain.

D01's answer, in SQL

It is September 2026, so last month is August. Accessories, 166.00, by hand on D01. Now as a query.

Suggested time · 22 minutes

[labs/D06/08-Evr-D01-Category-Revenue/README.md](#)

Two joins, nothing changed

12 rows and 487.00 before and after both joins: each line finds one order and one product.

```
SELECT p.category,  
       SUM(i.qty * i.price) AS revenue  
FROM D03.order_items AS i  
INNER JOIN D03.orders AS o  
  ON i.order_id = o.order_id  
INNER JOIN D03.products AS p  
  ON i.product_id = p.product_id  
WHERE o.status = 'completed'  
      AND o.order_date BETWEEN '2026-08-01' AND '2026-08-31'  
GROUP BY p.category  
ORDER BY revenue DESC;
```

Completed orders, by category

Same as D01. Without the status test, Shoes wins: the cancelled order again.

```
SELECT p.category,  
       SUM(i.qty * i.price) AS revenue  
FROM D03.order_items AS i  
INNER JOIN D03.orders AS o  
  ON i.order_id = o.order_id  
INNER JOIN D03.products AS p  
  ON i.product_id = p.product_id  
WHERE o.status = 'completed'  
      AND o.order_date BETWEEN '2026-08-01' AND '2026-08-31'  
GROUP BY p.category  
ORDER BY revenue DESC;
```

→ Accessories 166.00 · Shoes 136.50 · Outerwear 84.00

Break

 Back at 7:00 PM

After the break: the lab and a review of C1. Ask in your group channel.

Lab: D04's question, answered

Revenue by category on Nakheel, with an audit after every join.

Launch the lab

D04's open question on Nakheel: which category brings in the most revenue?

Suggested time • 5 minutes

[labs/D06/09-Lab-Category-Revenue/README.md](#)

Lab: Revenue by category, with a join audit

In this lab, you will answer D04's open question on Nakheel, with an audit table filled in after every join.

Suggested time

35 minutes

[labs/D06/09-Lab-Category-Revenue/README.md](#)

Lab: Revenue by category, with a join audit

CHECKPOINTS

C1 Revenue by category from order_items and products, audited before and after.

C2 Add orders, keep completed ones, check the total against orders alone.

C3 After class: completed revenue by category and month.

HINT

Write the joins and check the audit before adding WHERE and GROUP BY.

STRETCH

The same join on thelook_ecommerce, after a key-uniqueness check.

Time's up. Let's review.

The C1 audit

Solved: `labs/D06/09-Lab-Category-Revenue/Solved/category_revenue.sql` · after class

order_items JOIN products: nothing changed

Each line finds exactly one product. C1: 6 categories, Outerwear first with 81,920.50 (all orders).

	Rows	SUM(qty * price)
order_items	7,516	340,370.00
order_items JOIN products	7,516	340,370.00

CLOSE

What you can do now

- Say the grain after a join
- Run the Five Checks
- Write INNER and LEFT JOIN
- Audit and fix a total

NEXT • D07

Nakheel has 500 customers. How many have never ordered, and how would you find them?